TP3: Programación Orientada a Objetos (POO).

Rubín, Alexia

Link al repositorio de GitHub: github.com/alx1as

1). Registro de Estudiantes

a. Crear una clase E*studiante* con los atributos: nombre, apellido, curso, calificación.

Métodos requeridos: mostrarInfo(), subirCalificacion(puntos), bajarCalificacion(puntos).

Tarea: Instanciar a un estudiante, mostrar su información, aumentar y disminuir calificaciones.

Principal.java [-/A]

```java
package unidad3;
public class Principal {
    public static void main(String[] args) {
        //crear instancia de estudiante
        Estudiante est = new Estudiante();
        est.mostrarInfo();
        est.subirCalificacion(10);
        est.mostrarInfo();
        est.bajarCalificacion(5);
        est.mostrarInfo();
    }
}
```

Estudiante.java [-/A]

```java
public class Estudiante {
    private String nombre="Alexia";
    private String apellido="Rubin";
    private String curso="Programación II";
    private double calificacion=0;

    public void mostrarInfo() {
        System.out.println(nombre + " " + apellido + "\n "
                + curso + "\n Calificación: " + calificacion);
    }
    public void subirCalificacion(int puntos) {
        if(puntos>0) {
            calificacion+=puntos;
        }
    }
    public void bajarCalificacion(int puntos) {
        if (puntos>0) {
            calificacion-=puntos;
        }
    }
```

Output

PROGRAMACION_II - /home/alexia/NetBeansProjects/PROGRAMACION_II × UNIDAD03_POO (run) ×

```
Alexia Rubin
 Programación II
 Calificación: 0.0
Alexia Rubin
 Programación II
 Calificación: 10.0
Alexia Rubin
 Programación II
 Calificación: 5.0
```

2. Registro de Mascotas

a. Crear una clase Mascota con los atributos: nombre, especie, edad.

Métodos requeridos: mostrarInfo(), cumplirAnios().

Tarea: Crear una mascota, mostrar su información, simular el paso del tiempo y verificar los cambios.

Principal.java [-/M]

```java
package unidad3;
public class Principal {
    public static void main(String[] args) {
        //crear instancia de estudiante
        Estudiante est = new Estudiante();
        est.mostrarInfo();
        est.subirCalificacion(10);//+10
        est.mostrarInfo();
        est.bajarCalificacion(5); //-5
        est.mostrarInfo();

        //instancia mascota
        Mascota masc = new Mascota ();
        masc.mostrarInfo();
        masc.cumplirAnios();// +1
        masc.mostrarInfo();
    }
}
```

Mascota.java [-/A]

```java
package unidad3;
public class Mascota {
    private String nombre = "Apu";
    private String especie = "Cashi";
    private int edad = 1;

public void mostrarInfo() {
    System.out.println(nombre + "\nEspecie: "+especie + "\nEdad: "+edad);
}
public int cumplirAnios() {
    return edad +=1;
}
}
```

Output

PROGRAMACION_II - /home/alexia/NetBeansProjects/PROGRAMACION_II × UNIDAD03_POO (run) ×

```
Apu
Especie: Cashi
Edad: 1
Apu
Especie: Cashi
Edad: 2
```

3. **Encapsulamiento con la Clase Libro**

a. Crear una clase Libro con atributos privados: titulo, autor, añoPublicacion.

Métodos requeridos: Getters para todos los atributos. Setter con validación para añoPublicacion.

Tarea: Crear un libro, intentar modificar el año con un valor inválido y luego con uno válido, mostrar la información final

Principal.java [-/M]

```java
public class Principal {
    public static void main(String[] args) {
        /* //crear instancia de estudiante
         Estudiante est = new Estudiante();
         est.mostrarInfo();
         est.subirCalificacion(10);//+10
         est.mostrarInfo();
         est.bajarCalificacion(5); //-5
         est.mostrarInfo();

         //instancia mascota
         Mascota masc = new Mascota ();
         masc.mostrarInfo();
         masc.cumplirAnios();// +1
         masc.mostrarInfo();*/

        //Intancia libro
        Libro lib = new Libro();
        lib.setAñoPublicacion(-300); //inválido
        System.out.println(lib.getAñoPublicacion());
        lib.setAñoPublicacion(2005);
        System.out.println(lib.getAñoPublicacion());
    }
}
```

Libro.java

```java
public class Libro {
    private String titulo = "Nuestra parte de noche";
    private String autor="Mariana Enriquez";
    private int añoPublicacion=2019;

    public String getTitulo() {
        return titulo;
    }

    public String getAutor() {
        return autor;
    }

    public int getAñoPublicacion() {
        return añoPublicacion;
    }

    public void setAñoPublicacion(int añoPublicacion) {
        if(añoPublicacion>0)
        this.añoPublicacion = añoPublicacion;
    }
```

Output

UNIDAD03_POO (run) × PROGRAMACION_II - /home/alexia/NetBeansProjects/PROGRAMACION_II ×

```
run:
2019
2005
```

4. **Gestión de Gallinas en Granja Digital**

a. Crear una clase Gallina con los atributos: idGallina, edad, huevosPuestos.

Métodos requeridos: ponerHuevo(), envejecer(), mostrarEstado().

Tarea: Crear dos gallinas, simular sus acciones (envejecer y poner huevos), y mostrar su estado.

Principal.java [-/M] × Mascota.java ×

```java
package unidad3;
public class Principal {
    public static void main(String[] args) {


        //instancia gallina1:
         Gallina gal1 = new Gallina();
         gal1.envejecer(1);
         gal1.ponerHuevo(2);
         gal1.mostrarEstado();
         //gallina 2
         Gallina gal2 = new Gallina();
         gal2.envejecer(5);
         gal2.ponerHuevo(5);
         gal2.mostrarEstado();
    }

}
```

Gallina.java [-/A] ×

```java
public class Gallina {
    private int idGallina=0;
    private int edad=0;
    private int huevosPuestos=0;
    public void ponerHuevo(int num) {
        if (num>0) {
            huevosPuestos+=num; //pone huevos por parámetro
        }
    }
    public void envejecer(int años) {
        if(años>0) {
            edad+=años; //envejecer por parámetro
        }
    }
    public void mostrarEstado() { //mostrar info
    System.out.println("ID: " + idGallina);
    System.out.println("Edad: " + edad + " años");
    System.out.println("Huevos puestos: " + huevosPuestos);
    }
}
```

Output

UNIDAD03_POO (run) × PROGRAMACION_II - /home/alexia/NetBeansProjects/PROGRAMACION_II ×

```
ID: 0
Edad: 1 años
Huevos puestos: 2
ID: 0
Edad: 5 años
Huevos puestos: 5
BUILD SUCCESSFUL (total time: 0 seconds)
```

5. **Simulación de Nave Espacial:** Crear una clase NaveEspacial con los atributos: nombre, combustible.

Métodos requeridos: despegar(), avanzar(distancia), recargarCombustible(cantidad), mostrarEstado().

Reglas: Validar que haya suficiente combustible antes de avanzar y evitar que se supere el límite al recargar.

Tarea: Crear una nave con 50 unidades de combustible, intentar avanzar sin recargar, luego recargar y avanzar correctamente. Mostrar el estado al final.

NaveEspacial.java:

```java
public class NaveEspacial {
    private String nombre="Alexius";
    private int combustible=50;

    public void despegar() {
        if (combustible >0) {
            combustible-=1;//si es mayor a cero despega y resta 1u.
            System.out.println("Despegó correctamente. Combustible restante: " + combustible);
        } else {
            System.out.println("Combustible insuficiente");
        }
    }

    public void avanzar(int distancia) {
        if(combustible>=distancia) {
            System.out.println("Avanzó correctamente "+distancia + " kilómetros");
            combustible-=distancia;//resto el combustible proporcional a la distancia.
        } else {
            System.out.println("Debe cargar más combustible");
        }
    }

    public void recargarCombustible(int cantidad) {
        if(cantidad > 0 && combustible + cantidad <= 200) { //min>0 max 200 u. de combustible
            combustible+=cantidad;
        }
    }
    public void mostrarEstado() {
        System.out.println("Nombre de la nave: " + nombre + "\n Cantidad de combustible: " + combustible);
    }
}
```

```java
public class Principal {
    public static void main(String[] args) {

        //instancia nave
         NaveEspacial nave = new NaveEspacial();
         nave.avanzar(90); // no debería avanzar
         nave.recargarCombustible(100); // cargo 100
         nave.despegar(); //despega correct
         nave.avanzar(90); // ahora si
         nave.mostrarEstado();
    }

}
```

```
out - UNIDAD03_POO (run) ×
run:
Debe cargar más combustible
Despegó correctamente. Combustible restante: 149
Avanzó correctamente 90 kilómetros
Nombre de la nave: Alexius
 Cantidad de combustible: 59
BUILD SUCCESSFUL (total time: 0 seconds)
```